AMENDMENT v0.2 DESIGN ONLY

CourierLedger M1 Database Design Specification

Technical Correction Amendment v0.2

A targeted correction to the 36-page Database Design Specification v0.1. This amendment supersedes **only** the sections, tables, constraints and test scenarios it explicitly names. Everything else in v0.1 stands unchanged.

> **Scope guard.** This document contains no application code and no executable migrations. SQL fragments are illustrative design notation, marked ADAPT, exactly as in v0.1 Section E. No courier rate, fee or statement format is invented. V2.2.1 formulas, integer poisha, per-line half-up rounding, NULL versus verified zero, and the explicit D_c confirmation-plus-evidence rule are preserved without modification.

What this amendment does

- Resolves five mandatory corrections, CL-01 to CL-05.
- Addresses seven additional audit findings, CL-06 to CL-12.
- For every correction: the original problem, the exact change, the affected section and table, the constraint or transaction-level enforcement, an updated test scenario, and a resolved / still-open verdict.
- Ends with a change log and a final list of remaining blockers.

Reading rule

Where this amendment and v0.1 disagree, this amendment wins for the named item only. Where it is silent, v0.1 applies.

> **Gating - unchanged.** M1 production implementation remains gated behind the Day 7 and Day 14 market-validation gates, merchant-confirmed Steadfast and Pathao evidence, a Docker + PostgreSQL verification environment, and explicit production-readiness approval. This amendment authorises none of those steps.

Amendment at a glance

Twelve corrections. Nine are fully resolved at design level. Three leave a named decision open - deliberately, because closing them would require evidence or a policy choice this review does not have.

ID	Subject	Affected v0.1 locations	Verdict
CL-01	Overall status: scenario-only V_total	D.6, T-16, Appendix 1	Resolved
CL-02	Negative reported payout state model	C (statement_lines), D, OD-12	Design resolved; OD-12 still open
CL-03	Match candidate immutability	C (shipment_match_candidates), E	Resolved - new decision-history table
CL-04	Idempotency ownership	C (idempotency_keys, import_batches, reconciliation_runs)	Resolved
CL-05	Determinism: run_hash behaviour	C (reconciliation_runs), T-12	Resolved - deduplication
CL-06	Malformed tenant UUID	E (app_current_merchant_id)	Resolved
CL-07	Immutable matching decision history	C, E, I	Resolved (with CL-03)
CL-08	Per-basis fee-line DQ states	C (reconciliation_fee_lines), D	Resolved
CL-09	Multiple components of one fee category	C (reconciliation_fee_lines)	Resolved
CL-10	Settlement allocation and reversal integrity	C (settlement_allocations), D.10	Resolved
CL-11	Trust boundary of tenant context	E	Resolved as design; deployment check open
CL-12	Workspace deletion vs financial retention	G, J	Partially resolved; policy decision open

> **No architecture change without a reason.** Only two structural additions are proposed: one new table (`shipment_match_decisions`, CL-03/CL-07) and typed result-reference columns on `idempotency_keys` (CL-04). Every other correction is a constraint, state model, policy statement or test rewrite inside the existing design.

Conventions used in this amendment

- **Supersedes** - the v0.1 text named is replaced by the text given here.
- **Clarifies** - v0.1 was ambiguous; this amendment fixes one reading and rejects the other explicitly.
- **Adds** - new material with no v0.1 counterpart.

CL-01 - Overall status classification: scenario-only V_total produces open

1. Original problem

In v0.1 Section D.6, the status table row for a scenario-only V_{total} variance reads: contractual payout status *matched*, overall status *matched* (with the scenario finding still listed). Test T-16 repeats that expectation. That contradicts the approved P0 final status logic, in which a shipment whose only out-of-tolerance variance sits in a non-contractual, scenario comparison is not settled - it carries an open finding. v0.1 therefore presented a shipment with a real, unexplained scenario gap as fully matched, which is exactly the kind of status that hides a finding.

2. Exact proposed change

Supersedes the D.6 status table and its precedence paragraph. The corrected rule is:

- If any required value is NULL or a data-quality finding blocks exact calculation, overall status is `needs_review`.
- Else, if V_R, S or V_{cond} is non-zero outside tolerance, overall status is `mismatch`. Only these three - fee/tax, statement arithmetic and the contractual payout comparison - can produce a contractual payout mismatch.
- Else, if C is non-zero outside tolerance **or** V_{total} is non-zero outside tolerance on a `reported_collected` base, overall status is `open`.
- Else, if any finding is non-zero but within tolerance, overall status is `matched_with_tolerance`, with gross amounts still shown.
- Else, overall status is `matched`.

V_{total} remains scenario-only. It can move a result from *matched* to *open*; it can never move a result to *mismatch*, and it never changes contractual payout status, which is decided by V_{cond} alone. The scenario finding is still stored as its own row in `reconciliation_findings` with its gross amount.

Combination	Contractual payout status	Overall status (v0.2)
Scenario V_total non-zero only; V_cond = 0; C = 0	matched	**open** (was: matched)
Under-collection only (C non-zero); V_cond = 0	matched	open (unchanged)
V_R, S or V_cond non-zero outside tolerance	mismatch	mismatch (unchanged)
Any required value NULL	not_calculable or as calculated	needs_review (unchanged)

3. Affected section and table

Section D.6 (status table and precedence paragraph); Section I test T-16; Appendix 1 invariant traceability row for classification precedence. Tables: `reconciliation_results` (column `overall_status` - no type change) and `reconciliation_findings` (no schema change).

4. Constraint or transaction-level enforcement

No new column is needed. The enforcement is a single deterministic derivation, applied once at result creation and recorded with the result:

- A CHECK on `reconciliation_results.overall_status` keeps the closed five-value list: `matched`, `open`, `mismatch`, `needs_review`, `matched_with_tolerance`.
- A consistency CHECK (or deferred constraint trigger, ADAPT at implementation) rejects the impossible combination `overall_status = 'matched'` together with any stored finding on the same result whose amount is non-zero outside the run tolerance. A *matched* label over a live finding becomes unrepresentable, not merely discouraged.
- Because results are append-only, the derivation cannot be repaired in place afterwards; a corrected classification requires a new run, preserving the original.

5. Updated test scenario

T-16 (revised) - Classification precedence. Construct results with: (a) collection-only variance; (b) scenario-only V_{total} variance with $V_{cond} = 0$ and $C = 0$; (c) fee variance plus under-collection; (d) a financial variance plus missing data. Expected overall statuses: **open, open, mismatch, needs_review** respectively - case (b) changes from v0.1's *matched*. In every case all findings are present as separate rows, and contractual payout status is decided by V_{cond} alone. Add case (e): scenario-only V_{total} variance inside tolerance, expected `matched_with_tolerance` with the scenario finding listed.

6. Status

> **Resolved.** The P0 final status logic now takes precedence everywhere it appears in v0.1: D.6, T-16 and Appendix 1 are aligned, and no remaining example shows a scenario-only variance as *matched*.

CL-02 - Negative reported payout: one consistent state-to-value model

1. Original problem

v0.1 contradicts itself in three places on the same column. Section C specifies `statement_lines.reported_payout_poisha` with a non-negative CHECK, then notes in the same cell that a statement might legitimately report a negative net amount - which the CHECK would reject. The shared state-to-value rule says `present_valid` requires a value greater than 0, which would also reject a legitimate negative. Open decision OD-12 meanwhile leaves negative net payout possible. Three rules, three different answers, for a column that feeds P_r, S and V_{cond}.

2. Exact proposed change

Supersedes the state-to-value rule in Section C and the `reported_payout_poisha` constraint. The model separates two questions that v0.1 merged: *is the value present and parseable?* (the state) and *is its sign permitted for this field?* (a per-field sign rule).

State	Value rule (all amount/state pairs)
present_valid	Value NOT NULL. Zero is not stored under this state - an explicit zero uses `explicit_zero`. Sign is governed by the field's sign rule below.
explicit_zero	Value exactly 0.
column_absent / blank / invalid	Value NULL. Never converted to zero.

Field	Sign rule
Reported collected COD (R); every reported and calculated fee amount	Must be >= 0 when present. A negative value is `invalid`, produces a data-quality finding, and is stored as NULL - never as a negative amount and never as zero.
Reported payout (P_r) only	May be negative **only** when two conditions both hold: the merchant-confirmed rule version for that courier account explicitly permits a negative net payout for the statement type in question, and the statement line carries evidence for that semantics. Otherwise a negative P_r is `invalid` as above.

Where a negative P_r is permitted and stored, $S = P_r - (R - X_r)$ and $V_{cond} = S - V_R$ are computed with the signed value exactly as V2.2.1 defines; nothing is clamped. Where it is not permitted, P_r is NULL, so S, V_{cond} and V_{total} are NULL and the result is `needs_review` - the honest outcome, rather than a fabricated zero payout.

> **What this amendment does not do:** it does not assert that any of Steadfast, Pathao, Paperfly, RedX or eCourier ever reports a negative payout. No such behaviour is evidenced. The model makes the database *capable* of representing it under evidence, and incapable of representing it without evidence.

3. Affected section and table

Section C, table `statement_lines` (columns `reported_payout_poisha`, `reported_payout_state`); the shared state-to-value CHECK paragraph in Section C; Section D (NULL propagation into S and V_{cond}); Section J, OD-12 (wording clarified, decision unchanged).

4. Constraint or transaction-level enforcement

- State-to-value CHECKs per amount/state pair, revised to the table above.
- Non-negativity CHECKs retained on R and on every fee column.
- For P_r: a row-level CHECK of the form *value >= 0 OR negative_payout_permitted*, where `negative_payout_permitted` is a boolean copied into the statement line's snapshot from the confirmed rule version at import time - so the permission is part of the immutable import record, not a live lookup that could change under a historical row.
- The rule-version field that carries the permission may only be set on a rule version that has linked merchant-confirmed evidence in `courier_rule_evidence`; the existing evidence requirement is the gate.

5. Updated test scenario

T-15b (new) - Payout sign model. Four synthetic statement lines: (a) negative P_r with no permitting rule version - expected state `invalid`, value NULL, data-quality finding, result `needs_review`; (b) negative P_r with a permitting, evidence-backed rule version - expected signed value stored, S and V_{cond} computed signed, no clamping; (c) negative reported collected COD - expected `invalid` and NULL in all cases, including under a permitting rule version; (d) negative reported fee - expected `invalid` and NULL. T-15 itself is unchanged.

6. Status

> **Design resolved; OD-12 remains open.** The contradiction is removed: schema, state model and OD-12 now agree. Whether any real courier statement actually carries a negative net payout stays unresolved until redacted merchant statements confirm or refute it. Until then the default in practice is case (a): negative payouts are rejected as invalid, because no permitting evidence exists.

CL-03 - Shipment match candidates: immutable decision history

1. Original problem

Section C describes `shipment_match_candidates` as append-only *with a single decision update* - a contradiction in one sentence. Section E then grants append-only tables no UPDATE policy and no UPDATE grant at all. An implementer must break one of the two: either add an UPDATE path that the security design says must not exist, or make match decisions unrecordable. Match decisions are also financially sensitive - confirming a weak match is what attaches reported money to a shipment - so a mutable decision field with no history would be an audit gap even if the contradiction were resolved by allowing the update.

2. Exact proposed change

Supersedes the `shipment_match_candidates` description in Section C and removes the phrase *single decision update*. The design becomes two immutable parts:

- **`shipment_match_candidates` (unchanged role, now strictly insert-only).** One row per candidate considered: statement line, shipment, match basis (`strong_tracking` or `weak_order_ref` - never amount alone, per T-19), and the candidate's computed strength. No decision columns. No row is ever updated or deleted.
- **`shipment_match_decisions` (new table, append-only).** One row per decision event about one candidate: `id` UUID PK; `merchant_id` (tenant FK); `candidate_id` (composite FK to the candidate, same tenant); `decision` CHECK in (`confirmed`, `rejected`, `superseded`); `decided_by_user_id`; `decided_at` TIMESTAMPTZ; `reason_note` TEXT NULL; `supersedes_decision_id` NULL, self-referencing composite FK, set when a later decision replaces an earlier one.

The *current* decision for a candidate is derived, never stored mutably: it is the latest decision row for that candidate that has not itself been superseded. Reversal of a wrong confirmation is a new `superseded` plus `rejected` (or fresh `confirmed`) pair of rows - the original confirmation stays in history, attributable to its user and time. A statement line may be reconciled only against a candidate whose current derived decision is `confirmed`, or against a strong unique match under the V2.1 precedence rules; the run snapshot records the decision row IDs it relied on, so a later reversal does not silently rewrite what a historical run used.

3. Affected section and table

Section B (ERD sub-diagram for import/matching - add `shipment_match_decisions`); Section C (candidate table rewritten; new decision table added - 29 tables in total, up from 28); Section E (append-only table list).

4. Constraint or transaction-level enforcement

- Both tables sit in the v0.1 append-only class: RLS ENABLE + FORCE, SELECT and INSERT policies and grants only, **no UPDATE or DELETE policy and no such grant** for the runtime role. The CL-03 contradiction disappears because no decision ever needs an UPDATE.
- Composite foreign keys (`merchant_id, candidate_id`) prevent a decision in one workspace referencing a candidate in another.
- A partial unique index on `shipment_match_decisions (merchant_id, candidate_id) WHERE supersedes_decision_id IS NULL AND decision IN ('confirmed','rejected')` is **not** proposed: concurrent first decisions must both be recordable so the conflict is visible. Instead, a deferred constraint trigger (ADAPT) rejects a second *current* `confirmed` decision for candidates of the same statement line, forcing the second confirmer to supersede explicitly.
- Every decision insert also writes an `audit_logs` row in the same transaction (actor, candidate, decision, prior decision ID), so the audit trail and the decision history cannot diverge.

5. Updated test scenario

T-21 (new) - Match decision history. Create a weak candidate; confirm it; then supersede and reject it; then confirm a different candidate for the same line. Expected: four decision rows exist in order, the first confirmation is unchanged, the derived current decision is the final confirmation, an UPDATE or DELETE on either table as the runtime role affects zero rows or is denied, and a reconciliation run created between the first confirmation and its reversal still records the original decision row ID in its snapshot.

6. Status

> **Resolved.** Candidates and decisions are both immutable; Section E's no-UPDATE rule now holds without exception for the matching tables; confirmation and rejection are recorded, audited and linked to the candidate by foreign key. This is also the resolution of CL-07 (see below).

CL-04 - Idempotency: one authoritative architecture

1. Original problem

v0.1 spreads idempotency across three places without saying which one wins: key material on `import_batches`, key material on `reconciliation_runs`, and a standalone `idempotency_keys` table. A replayed request could therefore be deduplicated by one mechanism, missed by another, or - worst case - deduplicated against the wrong operation. v0.1 also leaves the link from a key to the object it created as an unenforced polymorphic reference (an operation label plus a bare ID), which no foreign key can check and no tenant boundary can protect.

2. Exact proposed change

Supersedes the idempotency parts of Sections C and H. There is now exactly one authoritative store:

- **`idempotency_keys` is the sole owner of idempotency.** Any idempotency columns on `import_batches` and `reconciliation_runs` are removed. Those tables record *what was created*; they never record *which request created it*.
- **Typed result references replace the polymorphic reference.** The table carries one nullable foreign key per supported operation - `result_import_batch_id` and `result_reconciliation_run_id` in M1 - each a composite FK (`merchant_id`, ID) to its target table, plus an `operation` CHECK in (`import_create`, `run_create`). A CHECK requires exactly the reference matching `operation` to be NOT NULL and all others NULL. Adding an operation later means adding a column and extending the CHECK - a deliberate, reviewable schema change, not a new string in an unchecked field.
- **Uniqueness:** UNIQUE (`merchant_id`, `operation`, `idempotency_key`). The same key string reused for a different operation is a different key; reused by a different workspace is a different key.
- **Request hash:** `request_hash` (SHA-256 of the canonical request, excluding the key itself) is stored at first use. A replay with the same key and the same hash returns the original result reference. A replay with the same key and a *different* hash is an explicit conflict error - never a silent return of the original, and never a second execution.
- **Lifecycle columns:** `status` CHECK in (`in_progress`, `completed`, `failed`); `created_at`; `completed_at` NULL; `expires_at` NOT NULL.

3. Affected section and table

Section C (table `idempotency_keys` rewritten; key columns removed from `import_batches` and `reconciliation_runs`); Section B ERD (supporting-records sub-diagram); Section H (repository pattern - idempotency is claimed through one shared component, not per-module code); Section I test T-10.

4. Constraint or transaction-level enforcement

- **Transaction boundary.** The key row is claimed in its own short transaction (INSERT with ON CONFLICT DO NOTHING semantics at design level) *before* the work transaction begins, with `status = in_progress`. The work transaction, on success, updates the key row to `completed` and sets the typed result reference - this is the one sanctioned UPDATE in the design, on a table that is explicitly **not** in the append-only class, with an UPDATE policy and grant scoped to the runtime role and tenant. If the work transaction rolls back, the key row is marked `failed` by a compensating update, and a retry with the same key and hash may re-execute; a retry with a different hash is still a conflict.
- **Concurrent claim:** the unique constraint is the arbiter. The losing transaction sees the existing row and behaves as a replay (same hash: wait-and-return or return in-progress status; different hash: conflict). No application-level check-then-insert is relied on.
- **Expiry:** keys expire at `expires_at` - recommended default 24 hours after creation for both M1 operations (a design default, adjustable per operation by configuration, not a business rule). Expired rows may be deleted by the migrator/maintenance role only, never by the runtime role; after expiry the same key string may be claimed fresh. Expiry never deletes the created import batch or run, which are ordinary retained records.
- **RLS:** `idempotency_keys` is tenant-owned, RLS ENABLE + FORCE; the composite result FKs make a cross-tenant result reference impossible.

5. Updated test scenario

T-10 (revised) - Idempotency. (a) Replay import-create and run-create with the same key and same hash: the original object is returned, exactly one object exists, and the key row's typed reference points at it. (b) Replay with the same key and a different hash: explicit conflict error, no new object. (c) Use the same key string for the other operation: treated as an independent key. (d) Attempt to set both result references, or the reference that does not match `operation`: rejected by CHECK. (e) Attempt a result reference to another workspace's batch: rejected by the composite FK. (f) Simulate a rolled-back work transaction: key is `failed`, and a same-hash retry re-executes exactly once. (g) An expired key string can be claimed again; the originally created object still exists.

6. Status

> **Resolved.** One table owns idempotency, references are relational and tenant-checked, and hash, retry, transaction and expiry behaviour are each specified once.

CL-05 - Determinism: identical inputs deduplicate to one run

1. Original problem

Section C places UNIQUE (`merchant_id`, `run_hash`) on `reconciliation_runs`, which makes a second identical run impossible. Test T-12 then allows *either* deduplication *or* two runs with identical values. A test that accepts both outcomes tests nothing, and the API contract is left undefined: a client retrying a run-create cannot know whether it will get the existing run, a new run, or a constraint error.

2. Exact proposed change

Clarifies in favour of the schema. The selected behaviour is **deduplication**:

- `run_hash` remains the hash of the canonical run inputs: input snapshot, rule snapshot, tolerance, and engine version - the same inputs that define determinism.
- UNIQUE (`merchant_id`, `run_hash`) stands. A run-create whose hash already exists for the workspace **returns the existing run** and creates nothing. This is the same contract as CL-04 idempotency, reached by content rather than by key: two different idempotency keys carrying byte-identical run inputs also resolve to the one existing run, with the second key's result reference pointing at it.
- Deliberately re-running after a correction is unaffected: corrected input data or a new rule version changes the hash, so a genuinely new run is created - which is also how v0.1's correction model (new runs, never edits) is supposed to work.
- There is no *force new run* escape hatch in M1. If a reviewer believes one is needed, that is a new open decision, not a silent flag.

3. Affected section and table

Section C (table `reconciliation_runs` - `run_hash` description); Section I test T-12; Section H (run-create repository behaviour). No schema change - the v0.1 unique constraint was already correct.

4. Constraint or transaction-level enforcement

- The unique constraint is the enforcement; the run-create path must translate a hash collision into *return existing*, not an error surfaced to the caller (design-level instruction; no code is specified here).
- Determinism itself is asserted on content: for identical inputs, the computed results, findings and fee lines of the single run are the canonical output. Any future engine change that alters output for identical stored inputs must change the engine version component of the hash, or historical comparability is lost - stated here as a release-process rule for M1, since the database cannot enforce it.

5. Updated test scenario

T-12 (revised) - Deterministic output and deduplication. Submit the same batch, rule version, tolerance and engine version twice, under two different idempotency keys. Expected: exactly **one** run exists; both requests resolve to its ID; its results, findings and fee lines are recorded once. Then change only the tolerance and resubmit: a second run is created with a different hash. The v0.1 wording permitting two identical runs is deleted.

6. Status

> **Resolved.** Schema, API expectation and test now state one behaviour: deduplication.

CL-06 - Malformed tenant UUID in app_current_merchant_id()

1. Original problem

Section E defines `app_current_merchant_id()` as `NULLIF(current_setting('app.merchant_id', true), '')::uuid`. That handles the unset and empty cases - both yield NULL, and policies fail closed. It does **not** handle a malformed non-empty value: the cast raises an error. The practical consequences are worse than a clean denial. Every policy evaluation on every tenant table errors, so a single bad setting turns a request into a 500-class failure rather than a zero-row denial; error text can leak internal detail; and a retry or health-check sharing the pooled connection inherits the failure mode. Fail-closed should mean *no rows*, not *an exception in every policy*.

2. Exact proposed change

Supersedes the helper definition in Section E.2. The function must return NULL for *any* value that is not a well-formed UUID - unset, empty, whitespace, truncated, or arbitrary text - and return the UUID only for a well-formed value. In design terms (ADAPT; not an executable migration):

```
-- ADAPT: validate format before casting; never cast unvalidated text.
-- Shape check first (8-4-4-4-12 hex), cast only on a match,
-- NULL otherwise. Equivalent exception-safe constructions are
-- acceptable if they return NULL on every malformed input and
-- never raise inside a policy.
```

Companion rules: the application must still set the context from an authenticated workspace resolution, never from an unvalidated client-supplied value (see CL-11); and a NULL return must be treated by the application as *no tenant context* - an authorisation failure for the request - not as a prompt to set a default tenant. There is no default tenant anywhere in the design.

3. Affected section and table

Section E.2 (tenant context initialisation and helper function); Section I tests T-02 and T-06 (missing-context and pool-leakage scenarios extended). No table changes.

4. Constraint or transaction-level enforcement

- Policies are unchanged: `merchant_id = app_current_merchant_id()` with a NULL right-hand side matches no rows for SELECT/UPDATE/DELETE and rejects INSERT via WITH CHECK - the existing fail-closed behaviour, now reached without an exception.
- The format validation happens inside the helper, so every policy on every tenant table inherits the fix at once; no per-table change is needed.

5. Updated test scenario

T-22 (new) - Malformed tenant context. For each of: unset setting, empty string, whitespace, `'not-a-uuid'`, a truncated UUID, and a valid UUID of a different workspace - attempt SELECT, INSERT, UPDATE and DELETE on a tenant table. Expected: the first five return zero rows / are rejected with **no exception raised by policy evaluation**; the sixth returns only that other workspace's rows where the session is legitimately scoped to it, and zero of the victim workspace's rows in all cases. Log output contains no unhandled cast error.

6. Status

> **Resolved.** Malformed context now fails closed as zero rows, consistently with missing context, and T-02/T-06 gain the malformed cases.

CL-07 - Immutable matching decision history (audit view)

1. Original problem

CL-07 is the audit-facing half of the CL-03 problem. Even if the candidate table's UPDATE contradiction were patched by allowing one controlled update, the *history* of who confirmed or rejected a match, when, and what they replaced would exist only in `audit_logs` - a general-purpose log whose detail payload is not relational. Reconstructing a match's decision history for a dispute would mean parsing log detail rather than querying linked rows, and nothing would guarantee the log and the decision could not diverge.

2. Exact proposed change

Resolved by the CL-03 design, recorded here so the audit requirement has its own traceable answer:

- Decision history is a first-class relational record: `shipment_match_decisions`, append-only, each row linked by composite FK to its candidate, carrying actor, timestamp, decision and the decision it supersedes (CL-03, item 2).
- The `audit_logs` row written in the same transaction is a *mirror* for cross-cutting audit queries, not the source of truth. If the two are ever found to differ, the decision table governs, because it is the constrained record.
- No decision is ever edited. A changed mind is a superseding row. The full chain - confirm, supersede, reject, confirm another candidate - is one query over one statement line's candidates and their decisions.

3. Affected section and table

Section C (new table `shipment_match_decisions`); Section C.7 / audit description (mirror wording); Section E append-only list; Section I test T-21 (shared with CL-03).

4. Constraint or transaction-level enforcement

As CL-03 item 4: no UPDATE/DELETE policy or grant on either matching table for the runtime role; decision insert and audit insert commit or roll back together in one transaction; composite FKs keep every decision inside its candidate's tenant.

5. Updated test scenario

T-21 (shared with CL-03), plus one audit assertion: after the confirm-supersede-reject sequence, the decision chain queried relationally and the mirrored audit rows name the same actors, decisions and order. Any mismatch fails the test.

6. Status

> **Resolved**, together with CL-03. No separate mechanism was added - one design answers both findings, which is why the amendment adds exactly one table rather than two.

CL-08 - Per-basis fee-line data-quality states

1. Original problem

Section C gives `reconciliation_fee_lines` a single `data_quality_state` per row. But a fee line holds three independently sourced amounts - expected-basis, reported-basis (recalculated) and reported-actual - and they fail independently. A statement can report a delivery fee perfectly (reported-actual known) while the reported-basis recalculation is impossible because D_c is unconfirmed, or the expected basis is unavailable because the rule is unknown. One state per row forces a choice between hiding which basis failed and marking the whole line defective, either of which breaks the D-section rule that X_e, X_R and X_r each become NULL exactly when *their own* contributing lines are unavailable.

2. Exact proposed change

Supersedes the fee-line state column in Section C. The single `data_quality_state` is replaced by three state columns, one per basis, each with the same closed CHECK list as v0.1 (`present_valid`, `explicit_zero`, `column_absent`, `blank`, `invalid`, `rule_unknown`, `contract_missing`):

- `expected_basis_state` - governs `expected_basis_poisha` and X_e.
- `reported_basis_state` - governs `reported_basis_poisha` and X_R. For an unconfirmed `cod_minus_delivery_fee` base this is `contract_missing` (with the `DQ_MISSING_CONTRACT` finding), while the other two bases may be fully valid.
- `reported_actual_state` - governs `reported_actual_poisha` and X_r; it mirrors the statement line's per-field state for that fee.

State-to-value consistency applies per basis, exactly as in CL-02's model: `present_valid` requires a NOT NULL amount for that basis; `explicit_zero` requires exactly 0; the remaining states require NULL for that basis only. The result-level totals follow per basis: X_e is the sum of expected-basis lines or NULL if any contributing expected-basis state is non-value; likewise X_R and X_r, independently. One basis being NULL never poisons the other two, and $V_R = X_r - X_R$ is NULL precisely when X_r or X_R is NULL - not merely because X_e is.

3. Affected section and table

Section C (table `reconciliation_fee_lines`); Section D (NULL propagation for X_e, X_R, X_r, V_R, S); Section I test T-15.

4. Constraint or transaction-level enforcement

- Three per-basis state-to-value CHECKs replace the single row-level CHECK.
- The v0.1 verification rule - a result's totals equal the sum of its fee lines in the matching basis, or are NULL - is restated per basis and remains a deferred constraint trigger / run-time assertion (ADAPT), since PostgreSQL cannot express it as a plain CHECK across rows.

5. Updated test scenario

T-15c (new) - Per-basis states. One fee line with: reported-actual present and valid; reported basis blocked by unconfirmed D_c; expected basis valid. Expected: `reported_actual_state = present_valid`, `reported_basis_state = contract_missing` with `reported_basis_poisha` NULL, `expected_basis_state = present_valid`; X_r and X_e are numbers, X_R is NULL, V_R is NULL, a `DQ_MISSING_CONTRACT` finding exists, and the result is `needs_review` - with no basis silently borrowing another basis's value.

6. Status

> **Resolved.** Each basis now carries its own truth about its own availability, which is what the V2.2.1 separation of bases requires at storage level.

CL-09 - Multiple components of the same fee category

1. Original problem

Section C keys `reconciliation_fee_lines` by (`result_id`, `fee_code`), with a closed fee-code list. A statement can legitimately carry two components of one category for one shipment - for example two separately stated delivery-related charges, or a fee and its separately stated surcharge under the same canonical category. The v0.1 key makes the second component uninsertable: it must be merged into the first (destroying per-line rounding evidence and the raw-to-normalised link) or dropped (understating X_r). Both workarounds violate approved rules - per-line half-up rounding, and preserving source rows without silently merging them.

2. Exact proposed change

Supersedes the fee-line key in Section C. The unique key becomes (`result_id`, `fee_code`, `component_seq`), where:

- `component_seq` INTEGER NOT NULL, CHECK >= 1, DEFAULT 1 - the component's order within its category for that result, assigned at run creation from the statement's own line order.
- `source_statement_field` TEXT NULL is added: the template-defined canonical field the component came from, so two components of one category remain distinguishable by provenance, not just by sequence.
- Each component is rounded independently (per-line half-up, unchanged), carries its own per-basis states (CL-08), and contributes to its basis total by summation across components. Totals are sums over components; nothing is pre-merged.
- The fee-code list stays closed. `component_seq` is not a route for inventing fee types: an unrecognised column is still preserved in raw rows with a data-quality warning and never folded into a total (v0.1 Appendix rule, unchanged).

3. Affected section and table

Section B ERD (fee-line entity - add `component_seq`); Section C (table `reconciliation_fee_lines` key and columns); Section D (per-line rounding statement clarified to per-component).

4. Constraint or transaction-level enforcement

- UNIQUE (`merchant_id`, `result_id`, `fee_code`, `component_seq`) replaces the two-column key; the composite-FK pattern to the parent result is unchanged.
- A run-creation assertion (ADAPT) checks that component sequences per (result, fee_code) are gap-free from 1 - a missing component is a creation error, not a silent gap in a total.

5. Updated test scenario

T-23 (new) - Fee components. A synthetic statement line with two delivery-category components, each requiring rounding. Expected: two fee-line rows with `component_seq` 1 and 2 and distinct source fields; each rounded independently half-up; X_r equals the sum of the two rounded components (and demonstrably differs, in the chosen fixture, from rounding the merged amount - proving no pre-merge occurred); a third insert reusing seq 2 is rejected by the unique key.

6. Status

> **Resolved.** Same-category components are representable without merging, dropping or widening the closed fee-code list.

CL-10 - Settlement allocation and reversal integrity

1. Original problem

v0.1's `settlement_allocations` mixes two models in one table: it is declared append-only with reversal by compensating row, yet carries an `is_active` boolean that a reversal would have to *update* - the same append-only contradiction as CL-03, in settlement this time. It also states the over-allocation rule (active allocations from one receipt never exceed the receipt amount) while relying on locking discipline plus a flagged deferred trigger, without defining the net-allocation computation that bank variance depends on, or what a reversal does to a batch that was previously fully allocated - including a batch whose bank variance was already computed.

2. Exact proposed change

Supersedes the allocation rules paragraph in Section C and clarifies Section D.10:

- **`is_active` is deleted.** Allocations are strictly append-only. An allocation row has `allocated_poisha` BIGINT NOT NULL, CHECK > 0, and a `kind` CHECK in (`allocation`, `reversal`). A reversal row references the allocation it reverses via `reverses_allocation_id` (composite self-FK, NOT NULL exactly when `kind = 'reversal'`, enforced by CHECK) and carries the same positive amount as the original; net allocation is always *allocations minus their reversals*, computed, never stored as a mutable flag.
- **Net totals are derived:** a receipt's net allocated total is the sum over its allocation rows minus the sum over their reversal rows; a batch's allocated total likewise. No stored total is authoritative.
- **Bank variance consequence:** bank variance is a stored value computed at the moment its preconditions hold (receipt explicitly complete AND batch fully allocated on net totals). If a later reversal makes the batch no longer fully allocated, that stored variance is **not** edited. Instead the batch's allocation status is recomputed as a new derived state and the previously computed variance is superseded in presentation by NULL - implemented at design level as: variance is only *reported* while its preconditions currently hold on net totals; the historical computation remains in the audit record. A re-completion after re-allocation computes a fresh variance. History is preserved; a stale variance is never presented as current.
- Allocations are never created by the system from an amount match alone, and never spread pro-rata (v0.1 rules, unchanged).

3. Affected section and table

Section C (table `settlement_allocations` - columns and rules); Section D.10 (bank variance preconditions restated on net totals); Section I test T-17.

4. Constraint or transaction-level enforcement

- CHECKs: amount > 0; `kind`/reversal-reference consistency; a reversal's amount equals its target's amount (deferred constraint trigger, ADAPT - a cross-row rule no plain CHECK can express).
- Uniqueness: at most one reversal per allocation - partial unique index on `reverses_allocation_id` WHERE `kind = 'reversal'`. Double reversal is unrepresentable.
- Over-allocation: enforced in the allocation transaction with a row lock on the receipt, checking the *net* total before insert, backed by the deferred constraint trigger v0.1 already flagged. The amendment makes the trigger mandatory rather than *reviewed as part of implementation*: without it the rule is a convention, not a constraint.
- Same-transaction audit row for every allocation and reversal.

5. Updated test scenario

T-17 (revised) - Partial settlement allocations and reversals. v0.1 steps retained (bank variance NULL at every intermediate step; variance only after completion plus full allocation). Added: reverse one allocation after full allocation and variance computation. Expected: the reversal is a new row, the original is unchanged, a second reversal of the same allocation is rejected, net allocated total falls, the batch no longer reports a current bank variance (reported as NULL) while the historical computation remains auditable, and an allocation that would push a receipt's net total above its amount is rejected.

6. Status

> **Resolved.** Allocations are now append-only without exception, reversals are single and linked, and the interaction between reversal and bank variance is defined instead of left to implementer judgement.

CL-11 - Trust boundary of application-set tenant context

1. Original problem

Section E's isolation model rests on `set_config('app.merchant_id', ...)` issued by the application. v0.1 specifies the mechanics carefully - transaction-local, set inside the working transaction, never session-level - but does not state the trust assumption out loud: **RLS cannot distinguish a context set for the authenticated workspace from a context set to any other workspace's ID by compromised or buggy application code.** Whoever can set the variable can become any tenant, within the runtime role's grants. Left unstated, a reviewer can over-trust RLS as a complete boundary and under-invest in the controls that actually guard the setting of the context.

2. Exact proposed change

Adds an explicit trust-boundary statement to Section E, and three design requirements that follow from it:

- **Stated boundary:** RLS is the enforcement layer against cross-tenant *queries* - application bugs, missing filters, injection into query construction. It is not a boundary against a fully compromised application process. That limitation is now written into the specification rather than left implicit.
- **Context is set exactly once per transaction, from server-side session state.** The merchant ID must come from the authenticated session's resolved workspace membership (checked against `merchant_memberships`), never from a request parameter, header or body field. Any code path that accepts a client-supplied workspace identifier and sets context from it without the membership check voids the model; this is named as a review checkpoint for every future endpoint.
- **Context-setting is audit-visible in aggregate:** the application logs (outside the tenant tables) the workspace context asserted per transaction at debug-to-audit level sufficient to detect anomalous switching - for example one user session asserting many different workspaces - without logging financial content.
- **Defence in depth retained from v0.1:** the runtime role is non-superuser and non-owner, FORCE RLS is on, composite FKs block cross-tenant references even if a row is somehow read, and migrator credentials never ship to the application. None of these is weakened.

No stronger database mechanism (for example per-tenant roles or per-tenant databases) is adopted: for a shared-schema modular monolith at M1 scale that would be the over-engineering v0.1 and the brief both warn against. The correction is honesty about the boundary plus the membership-check requirement, not a new architecture.

3. Affected section and table

Section E (new trust-boundary subsection, placed before the policy SQL so no reader reaches the SQL without it); Section A (risk register - the assumption moves from implicit to named); Section H (repository pattern - context set in one middleware/dependency point, nowhere else).

4. Constraint or transaction-level enforcement

- Single context-setting point: tenant context is set in exactly one application component through which every tenant-scoped transaction passes; repositories never set it themselves. This is a code-review and test enforcement, stated here as a design invariant.
- Membership check precedes context-setting in the same request flow: no membership row, no context, and CL-06's fail-closed NULL behaviour covers any gap downstream.

5. Updated test scenario

T-24 (new) - Context trust boundary. (a) Authenticate as a member of workspace A and submit requests carrying workspace B's ID in parameter, header and body positions in turn: expected - every response is scoped to A or rejected; no response contains B data; context is never asserted as B. (b) Authenticate as a user with no membership in any workspace: expected - no tenant rows are readable or writable. (c) Review assertion, recorded with the test: a static check confirms tenant context is set in exactly one named component of the codebase.

6. Status

> **Design resolved; deployment verification open.** The boundary is now stated and the membership-check requirement is specified. Whether a given deployment actually satisfies it can only be verified against built software and its configuration, which remain gated - it is carried into the blocker list as a production-readiness check, not claimed as done.

CL-12 - Workspace deletion and financial-history retention

1. Original problem

v0.1 Section G flags retention periods as policy/legal decisions - correctly - but leaves the *mechanics* of workspace deletion unspecified where they collide with the rest of the design: tenant foreign keys cascade or restrict in ways never stated for deletion; reconciliation runs, statement lines and audit logs are append-only and historically meaningful; and P0/V2.2.1 correction semantics depend on history surviving. A workspace-deletion request therefore has no defined answer in v0.1, and the two naive implementations are both wrong: hard cascade deletion destroys financial history (and breaks the append-only promise to every other record that references it), while refusing deletion outright ignores a legitimate merchant request and possible legal obligations.

2. Exact proposed change

Clarifies Section G with a two-stage model, while keeping every retention *period* an open policy decision:

- **Stage 1 - closure (immediate, fully specified).** Deleting a workspace first closes it: memberships are ended, sessions revoked, the workspace is marked `closed_at` (new nullable TIMESTAMPTZ on `merchants`), and it becomes read-only - no new imports, runs or settlements - while remaining readable by its former Owner for export. Closure is reversible only by an explicit, audited re-open during a grace window whose length is a policy decision (open, below).
- **Stage 2 - erasure (deferred, policy-gated).** Personal data and uploaded file bytes are erasable under the retention policy once set: user personal fields are anonymised (rows retained with identifiers replaced, so FK integrity survives), and `file_objects.purged_at` is set with stored bytes deleted, per v0.1. **Financial history is not hard-deleted by workspace erasure:** reconciliation runs, results, findings, fee lines, statement lines and settlement records are retained in anonymised-tenant form - merchant identity link severed, amounts and rule snapshots intact - for the financial retention period. That period is a legal/policy decision for Bangladesh that this amendment does not invent.
- **Ordering rule:** no erasure step may run while it would orphan or alter a reconciliation record inside its retention period; where a legal erasure duty and a legal retention duty genuinely conflict for the same record, the conflict is escalated as a policy decision (new OD-16, below) - the database design does not silently pick a winner.

3. Affected section and table

Section G (lifecycle - workspace deletion subsection rewritten; user deletion clarified to anonymise-not-delete where FKs require the row); Section C (table `merchants` - add `closed_at`); Section J (new OD-16).

4. Constraint or transaction-level enforcement

- All tenant FKs from financial tables to `merchants` are ON DELETE RESTRICT (v0.1 composite-FK pattern unchanged) - hard deletion of a merchant row with financial history is rejected by the database, which is the enforcement that makes Stage 2 anonymisation rather than deletion the only available path.
- Closure is enforced by RLS-adjacent write policies / status checks (ADAPT): INSERT policies on tenant tables additionally require the workspace to be open, so a closed workspace cannot accumulate new financial records even if application code attempts it.
- Anonymisation runs as a migrator-role maintenance operation, never under the runtime role, and writes an audit record of what was anonymised and when - without recording the erased values.

5. Updated test scenario

T-25 (new) - Workspace closure and retention. Create a workspace with imports, runs and settlements; close it. Expected: new import and run creation are rejected at the database level; historical runs remain readable and byte-identical; a hard DELETE of the merchant row is rejected by FK restriction. Then run the anonymisation procedure: expected - user personal fields are replaced, file bytes are purged with `purged_at` set, financial rows remain with amounts intact and the merchant identity link severed, and no reconciliation identity (V_{total} = C - delta - V_R + S) is affected on any retained result.

6. Status

> **Partially resolved.** The mechanics are now specified and enforceable. The decisions that remain open are genuinely not this document's to make: grace-window length, financial retention period, and the erasure-versus-retention conflict rule (OD-16) all require policy and legal review for Bangladesh. They are carried to the blocker list as non-blocking for design review, blocking for any production launch.

Consolidated test changes

Section I of v0.1 (T-01 to T-20) stands except as listed here. This table is the complete delta.

Test	Change	Source
T-10	Revised: single idempotency store, typed references, hash conflict, failed-transaction retry, expiry	CL-04
T-12	Revised: deduplication is the only expected outcome; two-identical-runs wording deleted	CL-05
T-15b	New: payout sign model, four cases	CL-02
T-15c	New: per-basis fee-line states	CL-08
T-16	Revised: scenario-only V_total case now expects **open**; tolerance case added	CL-01
T-17	Revised: reversal integrity, single-reversal rule, variance superseded by NULL after reversal	CL-10
T-21	New: match decision history and immutability	CL-03 / CL-07
T-22	New: malformed tenant UUID fails closed without exceptions	CL-06
T-23	New: multiple fee components per category	CL-09
T-24	New: tenant-context trust boundary	CL-11
T-25	New: workspace closure, anonymisation, retention	CL-12

Appendix 1 delta - invariant traceability

- Row *classification precedence*: enforcement now reads "D.6 as corrected by Amendment CL-01; consistency CHECK rejecting matched-over-live-finding"; test T-16 (revised).
- New row: *scenario variance never triggers contractual mismatch* - enforced by the D.6 derivation (CL-01); tests T-16, T-20.
- New row: *negative amounts only where evidenced* - per-field sign CHECKs and the rule-version permission snapshot (CL-02); test T-15b.
- Row *corrections create new records or runs*: now also covers match decisions (CL-03) and settlement reversals (CL-10); tests T-21, T-17.
- Row *idempotent creation*: single-store architecture (CL-04); test T-10.

> **No test is claimed to have passed.** These are design-stage scenarios for the future M1 test suite, exactly as in v0.1 Section I. Nothing in this amendment was executed against a database, because no M1 implementation exists and none is authorised.

Change log - v0.1 to v0.2 (this amendment)

Item	Type	Change
CL-01	Supersedes	D.6 status table, precedence paragraph, T-16, Appendix 1: scenario-only V_total outside tolerance now yields overall `open`.
CL-02	Supersedes	State-to-value model split from per-field sign rules; negative P_r possible only under evidenced rule permission; OD-12 clarified, still open.
CL-03 / CL-07	Supersedes + Adds	Candidates strictly insert-only; new append-only table `shipment_match_decisions` (29 tables total); T-21 added.
CL-04	Supersedes	`idempotency_keys` sole owner; key columns removed from import_batches and reconciliation_runs; typed result FKs replace polymorphic reference; T-10 revised.
CL-05	Clarifies	Deduplication on (`merchant_id`, `run_hash`) is the single specified behaviour; T-12 revised.
CL-06	Supersedes	`app_current_merchant_id()` returns NULL on any malformed value, never raises; T-22 added.
CL-08	Supersedes	Single fee-line DQ state replaced by three per-basis states with per-basis CHECKs; T-15c added.
CL-09	Supersedes	Fee-line key extended with `component_seq`; `source_statement_field` added; T-23 added.
CL-10	Supersedes	`is_active` deleted; allocation / reversal kinds; single-reversal index; mandatory deferred trigger; bank-variance supersession rule; T-17 revised.
CL-11	Adds	Trust-boundary statement in Section E; membership-checked, single-point context setting; T-24 added.
CL-12	Clarifies + Adds	Two-stage closure-then-anonymisation model; `merchants.closed_at` added; OD-16 opened; T-25 added.

Remaining blockers

Blocking any M1 implementation (unchanged from v0.1)

- **B-1.** Market validation: Day 7 and Day 14 gates not yet run; no interview evidence exists.
- **B-2.** No merchant-confirmed courier evidence for any of Steadfast, Pathao, Paperfly, RedX or eCourier - no rule version can yet be populated without inventing contract semantics, which is prohibited.
- **B-3.** Docker + PostgreSQL verification environment not established; no design SQL has been executed.
- **B-4.** Explicit production-readiness approval not given.

Open decisions carried forward

- **OD-12 (CL-02)** - whether any real courier statement reports a negative net payout. Open until redacted merchant statements answer it. The schema now supports either answer safely.
- **OD-16 (new, CL-12)** - workspace grace window, Bangladesh financial retention period, and the erasure-versus-retention conflict rule. Requires policy and legal review. Blocks production launch, not design review.
- **CL-11 deployment check** - the tenant-context trust boundary is specified; verifying a real deployment honours it is a production-readiness task.
- All other v0.1 open decisions OD-01 to OD-15 stand as written, except OD-12 as clarified above.

> **Final statement.** With this amendment applied, v0.1 has no known internal contradictions among the twelve audited items: nine are resolved at design level, and three (CL-02 / OD-12, CL-11 deployment verification, CL-12 / OD-16) remain open for named external reasons - evidence or policy - rather than design ambiguity. The specification is a coherent, reviewable blueprint. It is not an authorisation to build: M1 production implementation remains gated behind market validation and explicit production-readiness approval.